Lab: Parallel Overhead in Python (Before/After Performance Engineering)

Dr. Xiaoyuan Suo (xiaoyuansuo51@webster.edu)

Learning goals

By the end of this lab, you will be able to:

- Measure runtime correctly in Python (time.perf_counter, multiple trials, median).
- Explain why naïve multiprocessing can be slower than serial.
- Reduce overhead using:
  - coarser task granularity
  - chunksize
  - initializer/global worker state (avoid sending big data repeatedly)
- Compute and interpret speedup and efficiency.

What you will build

A program that computes a **weighted sum** over a large list:

$$\text{result} = \sum_{i=0}^{N-1} (a[i] \cdot i) mod 97$$

This is intentionally:

- CPU + memory work, but simple
- easy to split into chunks [start:end)

Part A — Baseline measurement (Serial) —do this part after class before the deadline

1. Run serial version for N = 8,000,000.
2. Record median runtime of 3 trials.

Deliverable: serial median runtime.

Part B — BEFORE (Naïve parallel that is often slower) —do this part after class before the deadline

You will parallelize in the *worst* reasonable way:

- Break into many tiny chunks
- Use chunksize=1 so the pool schedules tons of tasks
- Send the big list a as an argument to every task (pickling overhead)

1. Run naïve parallel with:

    - procs = 4 (or your machine's core count)
    - chunk_len = 10_000 (creates lots of tasks)
    - chunksize = 1
2. Record median runtime and compare to serial.

Expected observation: often slower than serial.

Deliverable: naïve parallel median runtime + explanation (2–4 sentences).

Part C — AFTER (Optimized parallel)—do this part during designated lab session

Fix the overhead using two changes:

1. Avoid repeatedly pickling large data
    - Use initializer to set the big list as a global in each worker
2. Reduce scheduling overhead
    - increase chunk_len (fewer tasks)
    - use a larger chunksize

Run optimized parallel with:

- chunk_len = 500_000
- chunksize = 8 (or 16)

Deliverable: optimized parallel median runtime + speedup/efficiency.

Part D — Compute speedup and efficiency—do this part during designated lab session

For the best parallel version you achieved:

$$S(p) = \frac{T_1}{T_p}, E(p) = \frac{S(p)}{p}$$

Deliverable: speedup and efficiency values.

Required submission (what students turn in)

1. The completed Python file (or your modified one).
2. A short report (bullet points ok) containing:

- Serial median time
- Naïve parallel median time + "why slower" (overhead)
- Optimized parallel median time
- Speedup and efficiency for optimized version

- 1–2 sentences: What overhead sources mattered most?

Grading (100 pts)

- Correct timing methodology (median-of-3, perf_counter): **20**
- Serial baseline correct: **10**
- Naïve parallel implemented + results recorded: **20**
- Optimized parallel implemented + results recorded: **25**
- Speedup + efficiency computed correctly: **15**
- Explanation quality (overhead sources): **10**